GOLD: Exceptions (Mon Oct 5)

Announcements

Midterm 1 Retakes

Retakes are Wednesday October 7 in the 6:30 - 7:30 pm or 8:00 - 9:00 pm window.

Seat assignments will be sent out tonight by email and posted to the Brightspace gradebook tomorrow.

We are having lecture Mon, Wed, and Fri this week. No cancellations during a retake week.

Instructor Office Hours This Week

Prof. Borkowski's office hours this week

- Tuesday 2:30 - 4:00 pm (as usual)
- **Wednesday 12:30 - 2:00 pm**
- **no Thursday office hours**

Casting (from Friday)

From Friday and relevant for today: a program that compiles, and then stops with an exception.

Casting objects

In one of the polls we looked at:

```
Payable p = new Coach("Knuth");     // line 1
IO.println(p.getName());            // line 3
```

We saw `p.getName()` did not compile. Why?

The object is a `Coach` and it has a name, but through a `Payable` variable we may call only what `Payable` promises.

Remember casting numbers? `(int) 8.99` is `8` .

The same syntax works on objects:

```
class Main {
    void main() {
        Payable p = new Coach("Knuth");
        Coach c = (Coach) p;             // a cast: "trust me, this Payable is a Coach
        IO.println(c.getName());
        IO.println(c == p);              // one object, two variables

        Payable q = new Building("Matthews");
        Coach d = (Coach) q;             // this compiles too. Is it true?
        IO.println(d.getName());
    }
}

interface Payable {
    String payTo();
    double weeklyPay();
}

class Person {
    private String name;
    Person(String name) { this.name = name; }
    String getName() { return name; }
}

class Coach extends Person implements Payable {
    Coach(String name) { super(name); }
    @Override public String payTo() { return getName(); }
    @Override public double weeklyPay() { return 2400.0; }
}

class Building implements Payable {
    private String name;
    Building(String name) { this.name = name; }
    @Override public String payTo() { return "Physical Facilities"; }
    @Override public double weeklyPay() { return 800.0; }
}
```

What we get:

```
Knuth
true
Exception in thread "main" java.lang.ClassCastException:
    class Building cannot be cast to class Coach
```

- A cast on an object **converts nothing**.

    - `(int) 8.99` made a new value;
    - `(Coach) p` **does not make a new object**.
    - `c` and `p` are two arrows to the same Coach.
    - The cast only gives us a variable of a different type, and the variable's type decides what we are allowed to ask.

- The compiler takes our word for it. Java **checks the promise when the program runs**, against the real object.
    - The first cast is true.
    - The second is not: a `ClassCastException`, and the program stops.

- Every `Coach` is a `Payable`, so `Payable p = new Coach(...)` needs no cast and cannot fail.
- The reverse needs a cast, and it can fail: not every `Payable` is a `Coach`.

Avoid crashes: ask first

We already know how to ask: `instanceof`, in the test-and-name form from `equals`.

```
class Main {
    void main() {
        Payable[] payroll = { new Coach("Knuth"), new Building("Matthews") };
        for (Payable p : payroll) {
            if (p instanceof Coach c) {              // is p a Coach? then call it c
                IO.println("coach: " + c.getName());
            } else {
                IO.println("not a coach: " + p.payTo());
            }
        }
    }
}

interface Payable {
    String payTo();
    double weeklyPay();
}

class Person {
    private String name;
    Person(String name) { this.name = name; }
    String getName() { return name; }
}

class Coach extends Person implements Payable {
    Coach(String name) { super(name); }
    @Override public String payTo() { return getName(); }
    @Override public double weeklyPay() { return 2400.0; }
}

class Building implements Payable {
    private String name;
    Building(String name) { this.name = name; }
    @Override public String payTo() { return "Physical Facilities"; }
    @Override public double weeklyPay() { return 800.0; }
}
```

```
coach: Knuth
not a coach: Physical Facilities
```

`p instanceof Coach c` is the question "is `p` a `Coach`?" and the cast in one step, so it cannot fail.

We suggest you use this style. A bare cast is for when we are certain what the object is.

Introduction

When programs go wrong

Every program we have written that reads a number has been carrying a grenade:

```
void main() {
    int burgers = Integer.parseInt(IO.readln("How many burgers? "));
    IO.println("Making " + burgers + " burgers.");
}
```

The user types `abc`:

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "abc"
    at java.base/java.lang.Integer.parseInt(Integer.java:588)
    ...
```

This program is correct Java, but it is one bad keystroke from death.

That's not a bug: the *input* is wrong. We can't fix the user.

What do we do when something goes wrong at runtime and it isn't a bug?

Today's tool: **exceptions**

The Fix: try/catch

The fix: `try` / `catch`

- Wrap the risky lines with `try`
- Say which failure you're prepared to handle with `catch`
- Say what to do about it in the `catch` block

```
void main() {
    try {
        int burgers = Integer.parseInt(IO.readln("How many burgers? "));
        IO.println("Making " + burgers + " burgers.");
    } catch (NumberFormatException e) {
        IO.println("That wasn't a whole number.");
        IO.println("(Java's complaint: " + e.getMessage() + ")");
    }

    IO.println("Program is still running!");
}
```

```
How many burgers? abc
That wasn't a whole number.
(Java's complaint: For input string: "abc")
Program is still running!
```

Instead of a crash, we produce a message a human can act on, and the program can continue.

(It gives up after one bad answer, though. More in a few minutes.)

try/catch Syntax

Anatomy of `try`/`catch`

```
try {
    risky-statements;                   // the code that might fail
} catch (NumberFormatException e) {     // WHICH failure, caught as an object
    recovery-statements;                // what to do about it
}
```

- If nothing goes wrong: the `catch` block is **skipped entirely**.
- If the named failure happens:
  - the rest of the `try` is **abandoned**,
  - the `catch` runs,
  - and the program **continues after** the whole statement (onto line 6).
- `e` is an object (like any other) carrying details: `e.getMessage()` is its human-readable text.
- `NumberFormatException` is a class (type) name, and `e` is like a method parameter

What Prints?

Consider:

```
void main() {
    try {
        IO.println("A");
        int n = Integer.parseInt("x1");
        IO.println("B");
    } catch (NumberFormatException e) {
        IO.println("C");
    }
    IO.println("D");
}
```

Pause and predict (poll time)

Q. What does this program print?

- **A.** `A`, `B`, `D`
- **B.** `A`, `C`, `D`
- **C.** `A`, `B`, `C`, `D`
- **D.** `A`, then a crash with the `NumberFormatException` trace

Exception Examples

Exceptions: Old Friends and New Friends

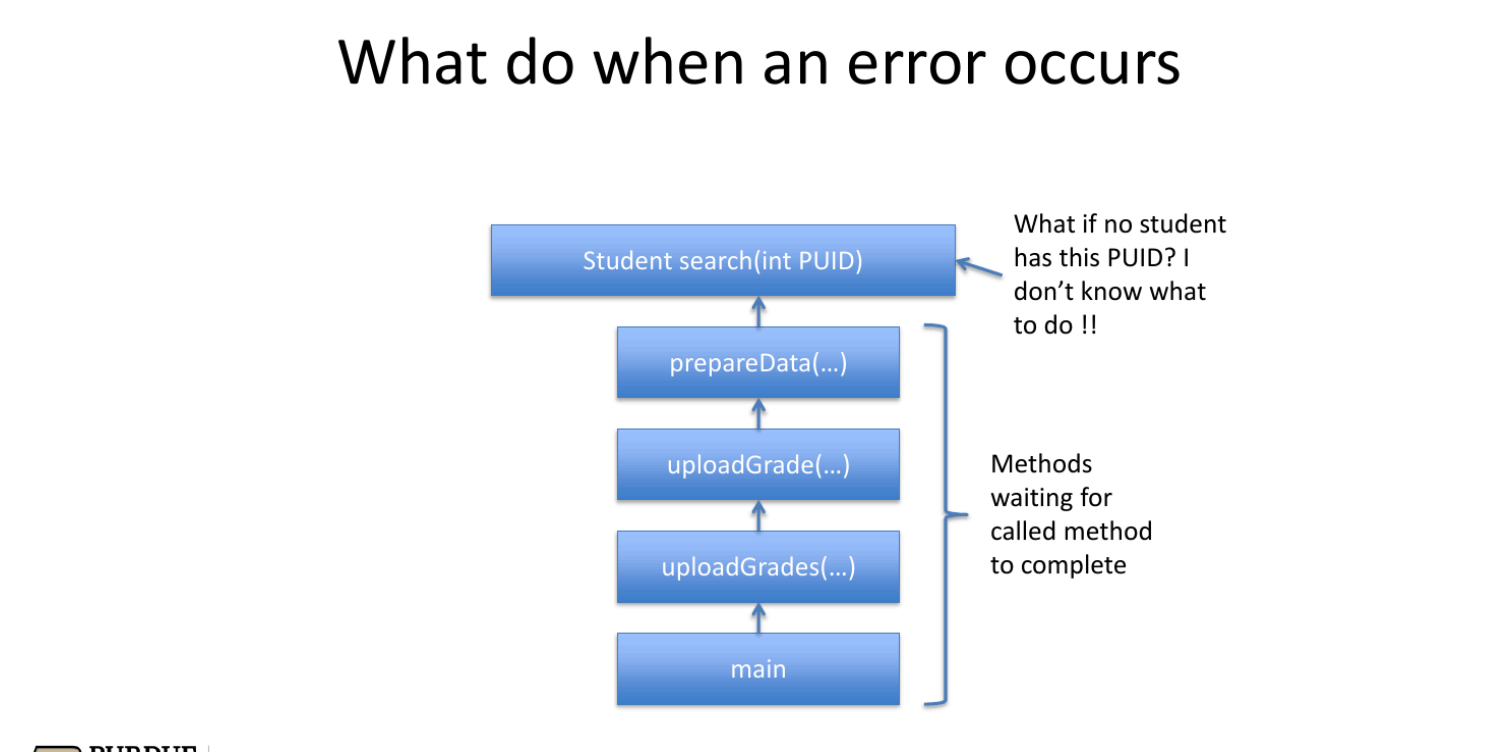
Every crash is an exception being thrown:

You wrote…	Java threw…
`a[5]` on a length-3 array	`ArrayIndexOutOfBoundsException`
`"hi".charAt(7)`	`StringIndexOutOfBoundsException`
`s.length()` when `s` is `null`	`NullPointerException`
`10 / 0`	`ArithmeticException`
`Integer.parseInt("abc")`	`NumberFormatException`
`(Coach) q` when `q` holds a `Building` (Friday)	`ClassCastException`

By contrast, compiler errors happen without Java ever running the code.

All About Crashes

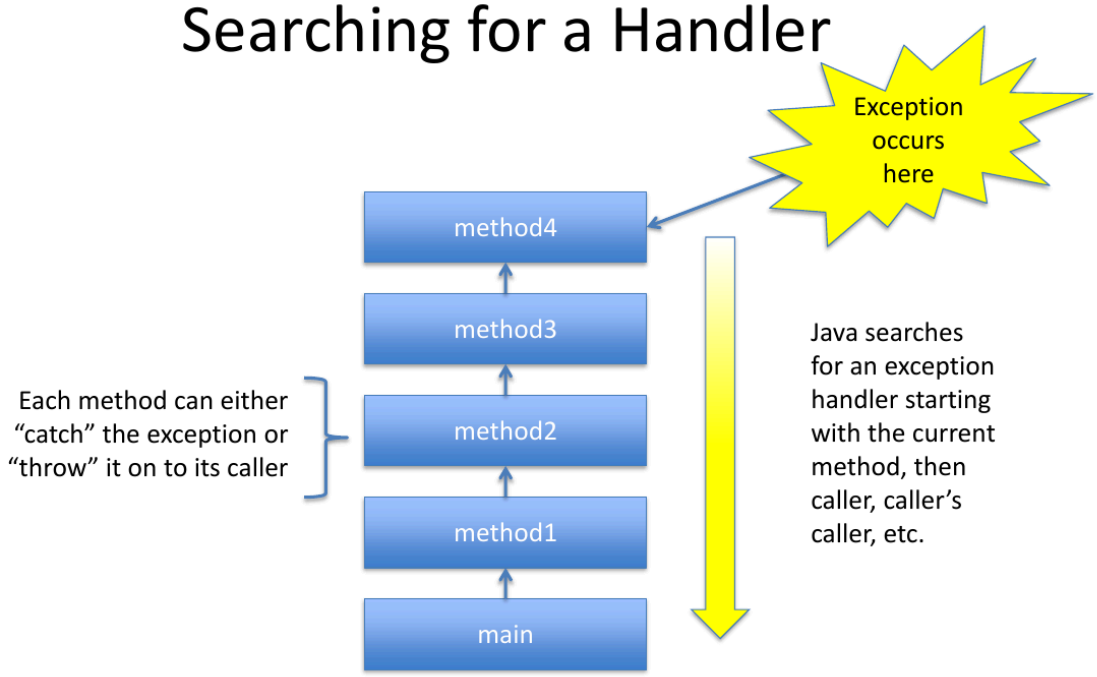
The story of a crash: who knows what to do?



The method that **detects** the problem is often the one *least able to fix it*.

`search` knows the student is missing. But only `main` knows whether to re-prompt, skip, or give up.

Throw, and the search for a handler



When an exception is **thrown**:

1. The current method stops where it is.
2. Java looks for a `catch` in the current method...
3. ...then in its **caller**, then the caller's caller... down the stack.
4. First matching `catch` wins. We continue after the `catch` (not necessarily the same method).
5. If none (not even in `main`) the program dies and prints the trace.

Let's see this live:

```
void main() {
    try {
        method1();
    } catch (ArithmeticException e) {
        IO.println("Handled in main");
    }
    IO.println("Done main");
}
void method1() {
    method2();
    IO.println("Done method1");
}
void method2() {
    method3();
    IO.println("Done method2");
}
void method3() {
    method4();
    IO.println("Done method3");
}
void method4() {
    // something bad happens
    int x = 10 / 0;
    IO.println("Done method4");
}
```

Example: Input Loop

The robust input loop

`parseInt` throws `NumberFormatException` on bad input, and so far every program we have written has ignored that.

Let's catch the exception and ask again:

```
void main() {
    int n = -1;
    boolean valid = false;
    while (!valid) {
        try {
            n = Integer.parseInt(IO.readln("How many burgers? "));
            valid = true;                       // only reached if parseInt succeeded
        } catch (NumberFormatException e) {
            IO.println("Not a whole number: try again.");
        }
    }
    IO.println("Making " + n + " burgers.");
}
```

```
How many burgers? abc
Not a whole number: try again.
How many burgers? 4.5
Not a whole number: try again.
How many burgers? 7
Making 7 burgers.
```

`valid = true` is reached only when `parseInt` succeeded

- a thrown exception abandons the rest of the `try`
- so the loop comes around and asks again.

This is the structure to use for every input loop that could crash.

Raising Your Own: throw

Raising your own: `throw`

Detecting nonsense is your code's job too:

```
class Main {
    void main() {
        IO.println(price(3));
        IO.println(price(-2));
    }

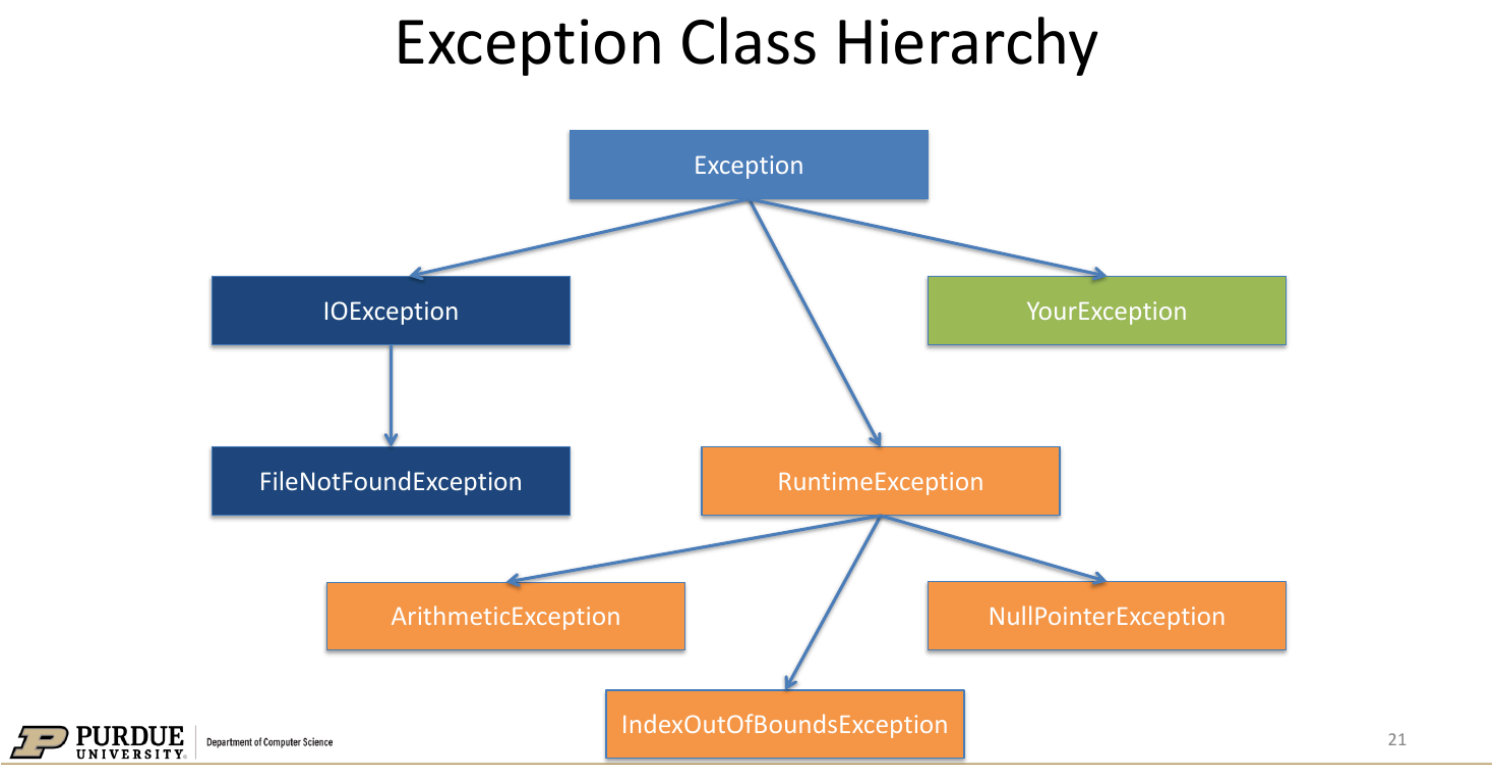
    int price(int qty) {
        if (qty <= 0) {
            throw new IllegalArgumentException("quantity must be positive, got " + qty
        }
        return qty * 5;
    }
}
```

```
15
Exception in thread "main" java.lang.IllegalArgumentException: quantity must be positi
    at Main.price(Main.java:9)
    at Main.main(Main.java:4)
```

- `throw` (no `s`): the statement that *raises* an exception, right here, right now.
- An exception is an object: you `new` it like any object, message included.
- Don't confuse `throw` with `throws` (with the `s`), which is covered later

The Family Tree: Checked and Unchecked

The family tree



Exceptions are classes, arranged in a hierarchy.

The family tree relation is "**is-a-kind-of**":

- A `FileNotFoundException` **is a kind of** `IOException`.
- An `ArithmeticException` **is a kind of** `RuntimeException`, which **is a kind of** `Exception`.

Why does the tree matter?

- **a `catch` for a type also catches every kind-of-it below.**
- `catch (IOException e)` catches `FileNotFoundException` too.

Checked vs. unchecked

Two important branches of the tree have two different compiler policies:

- **Unchecked**: `RuntimeException` and all its kinds / subclasses (`NullPointer…`, `ArrayIndexOutOfBounds…`, `NumberFormat…`, `Arithmetic…`):
    - usually indicate a **bug** (or bad data you can test for)
    - catch-or-declare **not required**
    - everything we've seen until now has been in this category
- **Checked:** everything else (`IOException`, `FileNotFoundException`, yours):
    - indicate **expected failures** of the outside world (missing file, broken connection)
    - the compiler **forces** you to catch or make a special declaration that you won't.
    - We'll meet lots of these
    - A class of your own that `extends Exception` is checked too, and demands the same (coming up).

Which Catch Runs?

Predict: which catch runs?

We can catch multiple exceptions: Java finds the first one (and **only** the first one) that matches:

```
void main() {
    try {
        int n = Integer.parseInt("0.0");
        IO.println(100 / n);
    } catch (ArithmeticException e) {
        IO.println("A: divided by zero");
    } catch (NumberFormatException e) {
        IO.println("B: that wasn't a number");
    } catch (Exception e) {
        IO.println("C: something else went wrong");
    }
}
```

Pause and predict (poll time)

Q. Which line prints?

- **A.** `A: divided by zero`
- **B.** `B: that wasn't a number`
- **C.** `C: something else went wrong`
- **D.** Nothing prints when the program crashes

Custom Exceptions

Your own exception type

An exception is a class, so you can extend one.

```
class Main {
    void main() {
        try {
            recordSale(40.00);
            recordSale(-25.00);
            recordSale(10.00);                  // never reached
        } catch (NegativeSaleException e) {
            IO.println("Refused: " + e.getMessage());
        }
    }

    void recordSale(double amount) throws NegativeSaleException {
        if (amount < 0) {
            throw new NegativeSaleException("sale of $" + amount);
        }
        IO.println("Recorded $" + amount);
    }
}

class NegativeSaleException extends Exception {
    NegativeSaleException(String message) { super(message); }
}
```

The body being almost empty is the point.

The name is the payload

Why bother, when `IllegalArgumentException` exists?

Because `catch` dispatches on **type**:

```
} catch (NegativeSaleException e) {   // catches YOUR failure only
```

- **The type is the message to the program;**

- `getMessage()` is the message to the *human*.
- A distinct type lets callers react to *your* failure specifically
  - no string-matching, no guessing.
- The constructor just passes the message up: `super(message)`

When you extend `Exception`

Your class is now **checked**: the compiler demands that every failure has an owner.

Throw it without answering the question "who handles this?":

```
error: unreported exception NegativeSaleException;
    must be caught or declared to be thrown
```

We have two choices:

- **Catch it:** `try`/`catch`, right there; or
- **Declare it:** pass the buck to your caller, in the method header:

```
void recordSale(double amount) throws NegativeSaleException {
    if (amount < 0) {
        throw new NegativeSaleException("sale of $" + amount);
    }
}
```

`throws` (with the `s`) is a *declaration*: "if this fails, my caller can have it."

And a method may pass more than one: list them with commas.

```
void recordSale(double amount, boolean open)
        throws NegativeSaleException, ClosedRegisterException {
    ...
}
```

Not every failure is **checked**. The earlier examples were all unchecked, so catching was optional.

If you want to create an unchecked exception, then extend `RuntimeException` instead.

One Word Changed

One word changed

Here is a `Deck` that refuses to deal from an empty deck.

Its exception class extends `RuntimeException`, and `deal()` neither catches nor declares:

```
class EmptyDeckException extends RuntimeException {
    EmptyDeckException(String message) { super(message); }
}

class Deck {
    private String[] cards = { "Ace of Spades" };
    private int size = 1;

    String deal() {                              // no throws clause, no try/catch
        if (size == 0) {
            throw new EmptyDeckException("no cards left");
        }
        return cards[--size];
    }
}
```

Pause and predict (poll time)

Q. What happens when we compile?

- **A.** Compile error: `unreported exception EmptyDeckException; must be caught or declared to be thrown`
- **B.** It compiles. An empty deck stops the program with `EmptyDeckException: no cards left`, unless some caller catches it
- **C.** It compiles, but `throw` has no effect on a `RuntimeException`
- **D.** Compile error: a class may not extend `RuntimeException`

Catch Order

Here's the same program from the earlier poll, with its catches reordered:

```
try {
    int n = Integer.parseInt(IO.readln("Number? "));
    IO.println(100 / n);
} catch (Exception e) {
    IO.println("something went wrong");
} catch (NumberFormatException e) {
    IO.println("that wasn't a number");
}
```

Pause and predict

Q. What happens?

- **A.** The more specific catch silently never runs
- **B.** Compile error: `exception NumberFormatException has already been caught`
- **C.** Both catches run, the broad one first
- **D.** Java reorders them automatically to run the `NumberFormatException` catch

Pattern Matching

We keep seeing Java code that asks "*is this thing of that kind?*" and, when the answer is yes, gives it a name:

where	**syntax**	**the question it asks**
in `equals` (and see the Casting slide)	`if (o instanceof Card c)`	is `o` a Card? Then call it `c`
after a `try`	`catch (NumberFormatException e)`	is the failure this kind? Then call it `e`
the general form	`case Coach c ->` inside a `switch`	is `p` a Coach? Then call it `c`

The general form:

```
class Main {
    String describe(Payable p) {
        return switch (p) {
            case Coach c    -> "coach " + c.getName();
            case Building b -> "a building";
            default         -> "something else that can be paid";
        };
    }

    void main() {
        Payable[] payroll = { new Coach("Knuth"), new Building("Matthews") };
        for (Payable p : payroll) {
            IO.println(describe(p) + ": " + p.weeklyPay());
        }
    }
}

interface Payable {
    String payTo();
    double weeklyPay();
}

class Person {
    private String name;
    Person(String name) { this.name = name; }
    String getName() { return name; }
}

class Coach extends Person implements Payable {
    Coach(String name) { super(name); }
    @Override public String payTo() { return getName(); }
    @Override public double weeklyPay() { return 2400.0; }
}

class Building implements Payable {
    private String name;
    Building(String name) { this.name = name; }
    @Override public String payTo() { return "Physical Facilities"; }
    @Override public double weeklyPay() { return 800.0; }
}
```

```
coach Knuth: 2400.0
a building: 800.0
```

- The cases are tried top to bottom and the first match wins, exactly like the catches.

- A broader type first is a compile error, exactly like the catches: `this case label is dominated by a preceding case label`.
- `default` is the catch-all for every kind we did not name.

What's different here?

Previously:

- the payroll loop chose the code by **override**
- the object decides,
- and a new class needs no edit to the loop.

With pattern matching:

- A switch on the type lets the **caller** decide,
- and a new operation needs no edit to the classes.

Best Practices

When *not* to catch

Two kinds of trouble:

	Expected failure	Bug
Examples	bad user input, an empty deck, a dropped connection	`a[i]` out of bounds, `null.length()`, negative quantity reaching `price()`
Right response	**catch:** re-prompt, default, friendly message	**let it crash:** read the trace, *fix the code*

Two habits that make debugging miserable:

- **The everything-catcher:**
  - `catch (Exception e)` around whole programs "so it stops crashing"
  - it doesn't fix anything,
  - it just stops *telling you*.

- **The empty catch block:** `catch (...) { }`
  - **Never leave a catch block empty**
  - it doesn't fix anything
  - it also just stops telling you
  - at minimum, print something!

Extra Reading: Two Designs

Extra reading: not presented in lecture.

Two designs: error codes vs. exceptions

Old style: reserve a value:

```
int dash = code.indexOf('-');   // returns -1 for "not found"
```

- Caller must remember to check, every time (forget → `substring(0, -1)` crash).
- A value must be *reserved*
- what value could `parseInt` reserve, when every `int` is a legal answer?

Exception style: the signal is out-of-band**:**

- failure travels a separate channel;
- it *cannot* be silently ignored;
- and no value is sacrificed.

Java's library splits this decision:

- Some methods return `null` for an error code (like no more data left)
  - expected, happens often
- `parseInt` throws an exception
  - exceptional event, bad input.

Take Home: The Box Office

The box office sells tickets at $12 each, and will sell at most 8 per person. `BoxOffice.java` and `Main.java` are both yours to complete in this exercise.

Part 1: Write a method that may refuse bad input.

- In `BoxOffice.java`, finish **`cost(int tickets)`**: `tickets` times `PRICE`.
- For a number below 1 or above `MAX_TICKETS`, do not return a number at all: `throw new IllegalArgumentException(...)` with a message that names the number, such as `tickets must be 1 to 8, got 12`.

Part 2: Write an input loop that never crashes. In `Main.java`, ask `How many tickets?` until you have a total to print:

- a reply that is not a whole number (`Integer.parseInt` throws `NumberFormatException`): print `That is not a whole number.` and ask again;
- a number that `cost` refuses: print `Sorry:` followed by the exception's message, and ask again;
- otherwise print `Total: $` followed by the cost, and stop.

A sample run, with the typed input in brackets:

```
How many tickets?
[abc]
That is not a whole number.
How many tickets?
[12]
Sorry: tickets must be 1 to 8, got 12
How many tickets?
[4]
Total: $48
```

That is two `catch` blocks on one `try`. If the compiler tells you one of them *has already been caught*, think back to today's last poll.

Run allows you to test the code interactively. Press **Test** to run the tests.